Linear Superposition in LLMs: Mechanistic Analysis

Motivation: We performed an experiment where we passed texts A and B through the LLM like this: LLM(A+B). Yes, by simply token-wise adding/averaging the embeddings for two separate texts. And we observed that even **before** any fine-tuning we have $LLM(A+B) \approx LLM(A) + LLM(B)$. We measured this by ranking the positions of the top-1 tokens from the clean single-stream run of both texts in the superposed-stream output distribution.

Surprisingly, we've found:

```
--- METRICS AGAINST VANILLA MODEL'S PREDICTION ---
VANILLA MODEL'S TOP TOKEN RANK (on non-missed vanilla targets):
  Text A: 10.14 ± 2.21
  Text B: 10.19 ± 2.03
  Average: 10.15 ± 1.39
```

Which is quite peculiar (the 10th position is quite low, considering that the Vocabulary size is often >50k), and it means that LLMs are somewhat linear.

In this notebook, we aim to investigate why Attention doesn't destroy the superposition of embeddings.

Core Question: When we feed $X_{mix} = \text{Norm}(Embed(A) + Embed(B))$ into a transformer, why does Attention (with its nonlinear Softmax) not completely destroy the superposition?

Experiments:

1. *Signal-to-Interference Ratio (SIR)* in Attention Scores
2. "*Ghost Context*" *Analysis* (Leakage via Values)
3. *Attention Pattern Overlap* vs. Quality degradation

In [1]:
```
import torch
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt
from transformers import AutoModelForCausalLM, AutoTokenizer
from typing import Dict, List, Tuple, Optional
from dataclasses import dataclass
from tqdm.auto import tqdm
import seaborn as sns
from collections import defaultdict

# Set style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

# Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cuda
```

Configuration

Choose between baseline model or fine-tuned dual-stream model.

In [2]:
```
# ============== CONFIGURATION ==============
# Choose model size (smaller for testing, larger for real analysis)
MODEL_NAME = "EleutherAI/pythia-6.9b"  # Options: pythia-410m, pythia-1.4b, pythia-6.9b

# Set to True to use a fine-tuned DualStreamLLM checkpoint
USE_FINETUNED = False
CHECKPOINT_PATH = None  # e.g., "./checkpoints/run_xxx/checkpoint_step_10000.pt"

# Test texts for analysis
TEXT_A = "The capital of France is Paris, which is known for the Eiffel Tower"
TEXT_B = "Machine learning algorithms can process large amounts of data efficiently"

# Analysis parameters
MAX_LENGTH = 64
NUM_SAMPLES_FOR_STATS = 20  # For statistical analysis across multiple text pairs
```

In [3]:
```
# Load model and tokenizer
print(f"Loading model: {MODEL_NAME}")
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    dtype=torch.bfloat16,
    device_map="auto",
    output_attentions=True,
    output_hidden_states=True,
)
model.eval()

# Get model config
config = model.config
num_layers = config.num_hidden_layers
num_heads = config.num_attention_heads
hidden_size = config.hidden_size
head_dim = hidden_size // num_heads

print(f"Model loaded: {num_layers} layers, {num_heads} heads, hidden_size={hidden_size}")
```

```
Loading model: EleutherAI/pythia-6.9b
The following generation flags are not valid and may be ignored: ['output_attentions', 'output_hidden_state
s']. Set `TRANSFORMERS_VERBOSITY=info` for more details.
Loading checkpoint shards:   0%|          | 0/2 [00:00<?, ?it/s]
Model loaded: 32 layers, 32 heads, hidden_size=4096
```

In [4]:
```
# Optional: Load fine-tuned DualStreamLLM
dual_stream_model = None

if USE_FINETUNED and CHECKPOINT_PATH:
    from model import DualStreamLLM

    print(f"Loading fine-tuned model from: {CHECKPOINT_PATH}")
    dual_stream_model = DualStreamLLM(
        model_name=MODEL_NAME,
        freeze_backbone=True,
        objective='separate',
        shared_embeddings=False,
    )

    checkpoint = torch.load(CHECKPOINT_PATH, map_location='cpu')
    dual_stream_model.load_state_dict(checkpoint['model_state_dict'])
    dual_stream_model = dual_stream_model.to(device)
    dual_stream_model.eval()
    print("Fine-tuned model loaded!")
else:
    print("Using baseline (non-finetuned) model for analysis")
```

```
Using baseline (non-finetuned) model for analysis
```

Utility Functions

Helper functions for extracting internal states and computing metrics.

In [5]:
```python
@dataclass
class AttentionInternals:
    """Stores Q, K, V and attention weights from a forward pass."""
    queries: List[torch.Tensor]  # [num_layers] each [batch, num_heads, seq_len, head_dim]
    keys: List[torch.Tensor]
    values: List[torch.Tensor]
    attention_weights: List[torch.Tensor]  # [num_layers] each [batch, num_heads, seq_len, seq_len]
    hidden_states: List[torch.Tensor]  # [num_layers+1] each [batch, seq_len, hidden_dim]
    logits: torch.Tensor  # [batch, seq_len, vocab_size]


@dataclass
class ForwardPassResult:
    """Complete result of a forward pass with all internals."""
    input_ids: torch.Tensor
    attention_mask: torch.Tensor
    internals: AttentionInternals
```

In [6]:
```python
def get_attention_layers(model):
    """Get list of attention layer modules (works for Pythia/GPT-NeoX)."""
    if hasattr(model, 'gpt_neox'):
        # Pythia architecture
        return [layer.attention for layer in model.gpt_neox.layers]
    elif hasattr(model, 'model') and hasattr(model.model, 'layers'):
        # LLaMA architecture
        return [layer.self_attn for layer in model.model.layers]
    elif hasattr(model, 'transformer'):
        # GPT-2 architecture
        return [layer.attn for layer in model.transformer.h]
    else:
        raise ValueError(f"Unknown model architecture: {type(model)}")


def extract_qkv_with_hooks(model, input_ids, attention_mask):
    """
    Extract Q, K, V matrices from all layers using forward hooks.

    Returns:
        ForwardPassResult with all internals
    """
    queries = []
    keys = []
    values = []
    handles = []

    attention_layers = get_attention_layers(model)

    def make_qkv_hook(layer_idx):
        def hook(module, input, output):
            # For Pythia/GPT-NeoX, the attention module computes QKV internally
            # We need to recompute Q, K, V from the hidden states
            hidden_states = input[0]  # [batch, seq_len, hidden_dim]

            # Get the QKV projection weights
            if hasattr(module, 'query_key_value'):
                # Pythia: fused QKV projection
                qkv = module.query_key_value(hidden_states)
                # Split into Q, K, V
                # Shape: [batch, seq_len, 3 * num_heads * head_dim]
                batch_size, seq_len, _ = qkv.shape
                qkv = qkv.view(batch_size, seq_len, num_heads, 3, head_dim)
                q = qkv[:, :, :, 0, :].transpose(1, 2)  # [batch, heads, seq, head_dim]
                k = qkv[:, :, :, 1, :].transpose(1, 2)
                v = qkv[:, :, :, 2, :].transpose(1, 2)
            elif hasattr(module, 'q_proj'):
                # LLaMA style: separate projections
                q = module.q_proj(hidden_states)
                k = module.k_proj(hidden_states)
                v = module.v_proj(hidden_states)
                batch_size, seq_len, _ = q.shape
                q = q.view(batch_size, seq_len, num_heads, head_dim).transpose(1, 2)
                k = k.view(batch_size, seq_len, num_heads, head_dim).transpose(1, 2)
                v = v.view(batch_size, seq_len, num_heads, head_dim).transpose(1, 2)
            else:
                raise ValueError("Unknown attention module structure")

            queries.append(q.detach().cpu())
            keys.append(k.detach().cpu())
            values.append(v.detach().cpu())
```

```
        return hook

    # Register hooks
    for layer_idx, attn_layer in enumerate(attention_layers):
        handle = attn_layer.register_forward_hook(make_qkv_hook(layer_idx))
        handles.append(handle)

    # Forward pass
    with torch.no_grad():
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            output_attentions=True,
            output_hidden_states=True,
            return_dict=True
        )

    # Remove hooks
    for handle in handles:
        handle.remove()

    # Extract attention weights and hidden states
    attention_weights = [attn.detach().cpu() for attn in outputs.attentions]
    hidden_states = [h.detach().cpu() for h in outputs.hidden_states]
    logits = outputs.logits.detach().cpu()

    internals = AttentionInternals(
        queries=queries,
        keys=keys,
        values=values,
        attention_weights=attention_weights,
        hidden_states=hidden_states,
        logits=logits
    )

    return ForwardPassResult(
        input_ids=input_ids.cpu(),
        attention_mask=attention_mask.cpu(),
        internals=internals
    )
```

In [7]:

```
def create_mixed_embeddings(model, input_ids_A, input_ids_B):
    """
    Create mixed embeddings: Norm(Embed(A) + Embed(B))
    Using dynamic rescaling to preserve norm.
    """
    # Get embedding layer
    if hasattr(model, 'gpt_neox'):
        embed_layer = model.gpt_neox.embed_in
    elif hasattr(model, 'model') and hasattr(model.model, 'embed_tokens'):
        embed_layer = model.model.embed_tokens
    elif hasattr(model, 'transformer'):
        embed_layer = model.transformer.wte
    else:
        raise ValueError("Unknown model architecture")

    with torch.no_grad():
        embed_A = embed_layer(input_ids_A)  # [batch, seq, hidden]
        embed_B = embed_layer(input_ids_B)

        # Sum embeddings
        sum_embeds = embed_A + embed_B

        # Dynamic rescaling (same as in model.py)
        norm_A = torch.norm(embed_A, p=2, dim=-1, keepdim=True)
        norm_B = torch.norm(embed_B, p=2, dim=-1, keepdim=True)
        norm_sum = torch.norm(sum_embeds, p=2, dim=-1, keepdim=True)

        target_norm = (norm_A + norm_B) / 2.0
        scale_factor = target_norm / (norm_sum + 1e-6)

        mixed_embeds = sum_embeds * scale_factor

    return mixed_embeds, embed_A, embed_B


def forward_with_embeddings(model, inputs_embeds, attention_mask):
    """Run forward pass with custom embeddings instead of input_ids."""
    queries = []
    keys = []
    values = []
    handles = []

    attention_layers = get_attention_layers(model)
```

```
    def make_qkv_hook(layer_idx):
        def hook(module, input, output):
            hidden_states = input[0]

            if hasattr(module, 'query_key_value'):
                qkv = module.query_key_value(hidden_states)
                batch_size, seq_len, _ = qkv.shape
                qkv = qkv.view(batch_size, seq_len, num_heads, 3, head_dim)
                q = qkv[:, :, :, 0, :].transpose(1, 2)
                k = qkv[:, :, :, 1, :].transpose(1, 2)
                v = qkv[:, :, :, 2, :].transpose(1, 2)
            elif hasattr(module, 'q_proj'):
                q = module.q_proj(hidden_states)
                k = module.k_proj(hidden_states)
                v = module.v_proj(hidden_states)
                batch_size, seq_len, _ = q.shape
                q = q.view(batch_size, seq_len, num_heads, head_dim).transpose(1, 2)
                k = k.view(batch_size, seq_len, num_heads, head_dim).transpose(1, 2)
                v = v.view(batch_size, seq_len, num_heads, head_dim).transpose(1, 2)
            else:
                raise ValueError("Unknown attention module structure")

            queries.append(q.detach().cpu())
            keys.append(k.detach().cpu())
            values.append(v.detach().cpu())
        return hook

    for layer_idx, attn_layer in enumerate(attention_layers):
        handle = attn_layer.register_forward_hook(make_qkv_hook(layer_idx))
        handles.append(handle)

    # Get the transformer core
    if hasattr(model, 'gpt_neox'):
        transformer = model.gpt_neox
    elif hasattr(model, 'model'):
        transformer = model.model
    else:
        transformer = model.transformer

    with torch.no_grad():
        # For Pythia, we need to bypass the embedding and feed directly
        outputs = model(
            inputs_embeds=inputs_embeds,
            attention_mask=attention_mask,
            output_attentions=True,
            output_hidden_states=True,
            return_dict=True
        )

    for handle in handles:
        handle.remove()

    attention_weights = [attn.detach().cpu() for attn in outputs.attentions]
    hidden_states = [h.detach().cpu() for h in outputs.hidden_states]
    logits = outputs.logits.detach().cpu()

    return AttentionInternals(
        queries=queries,
        keys=keys,
        values=values,
        attention_weights=attention_weights,
        hidden_states=hidden_states,
        logits=logits
    )
```

In [8]:

```
def tokenize_text(text: str, pad_to_length: int = None) -> Tuple[torch.Tensor, torch.Tensor]:
    """Tokenize text and return input_ids and attention_mask on device.

    Args:
        text: Input text to tokenize
        pad_to_length: If specified, pad/truncate to this exact length
    """
    if pad_to_length is not None:
        encoded = tokenizer(
            text,
            max_length=pad_to_length,
            padding='max_length',
            truncation=True,
            return_tensors='pt'
        )
    else:
        # No padding - just tokenize
        encoded = tokenizer(
```

```
            text,
            max_length=MAX_LENGTH,
            truncation=True,
            return_tensors='pt'
        )
    return encoded['input_ids'].to(device), encoded['attention_mask'].to(device)


def tokenize_pair_to_min_length(text_a: str, text_b: str) -> Tuple[torch.Tensor, torch.Tensor, torch.Tens
    """
    Tokenize two texts and truncate BOTH to the MINIMUM length.
    This ensures every position has a true superposition (no PAD + real token).

    Returns:
        input_ids_A, attention_mask_A, input_ids_B, attention_mask_B, common_length
    """
    # First tokenize without padding to get actual lengths
    enc_a = tokenizer(text_a, max_length=MAX_LENGTH, truncation=True, return_tensors='pt')
    enc_b = tokenizer(text_b, max_length=MAX_LENGTH, truncation=True, return_tensors='pt')

    len_a = enc_a['input_ids'].shape[1]
    len_b = enc_b['input_ids'].shape[1]

    # Use minimum length for clean experiment
    common_length = min(len_a, len_b)

    # Truncate both to common length
    input_ids_A = enc_a['input_ids'][:, :common_length].to(device)
    input_ids_B = enc_b['input_ids'][:, :common_length].to(device)

    # Attention masks are all 1s since no padding
    attention_mask_A = torch.ones_like(input_ids_A).to(device)
    attention_mask_B = torch.ones_like(input_ids_B).to(device)

    return input_ids_A, attention_mask_A, input_ids_B, attention_mask_B, common_length


def get_lm_head_weights(model):
    """Get the language model head weights for logit lens."""
    if hasattr(model, 'embed_out'):
        # Pythia
        return model.embed_out.weight.detach()
    elif hasattr(model, 'lm_head'):
        return model.lm_head.weight.detach()
    else:
        raise ValueError("Cannot find LM head")


def logit_lens(hidden_state: torch.Tensor, lm_head_weights: torch.Tensor, top_k: int = 10):
    """
    Project hidden state to vocabulary space.

    Args:
        hidden_state: [seq_len, hidden_dim] or [hidden_dim]
        lm_head_weights: [vocab_size, hidden_dim]
        top_k: number of top tokens to return

    Returns:
        List of (token_id, token_str, logit) tuples
    """
    if hidden_state.dim() == 1:
        hidden_state = hidden_state.unsqueeze(0)

    # Project: [seq_len, hidden] @ [hidden, vocab] = [seq_len, vocab]
    logits = hidden_state.float() @ lm_head_weights.float().T

    results = []
    for pos in range(logits.shape[0]):
        pos_logits = logits[pos]
        top_values, top_indices = torch.topk(pos_logits, top_k)
        pos_results = []
        for val, idx in zip(top_values, top_indices):
            token_str = tokenizer.decode([idx.item()])
            pos_results.append((idx.item(), token_str, val.item()))
        results.append(pos_results)

    return results if len(results) > 1 else results[0]
```

Experiment 1: Signal-to-Interference Ratio (SIR) in Attention Scores

Question: When we mix embeddings, does the Query from the mixed run still recognize its "native" keys?

Method:

1. Run clean passes: A alone, B alone, Mixed A+B
2. Extract Q_{mix} from mixed pass
3. Compare: $Q_{mix} \cdot K_A^T$ (Signal) vs $Q_{mix} \cdot K_B^T$ (Interference)
4. Compute SIR = Signal / Interference across layers

In [9]:
```
# Tokenize texts - truncate to MINIMUM length for clean experiment
# This ensures every position has true superposition (real_token_A + real_token_B)
input_ids_A, attention_mask_A, input_ids_B, attention_mask_B, common_length = tokenize_pair_to_min_length(

# For mixed pass, attention mask is same (all 1s, no padding)
attention_mask_mixed = attention_mask_A  # Both are identical (all 1s)

print("Text A:", TEXT_A)
print("Text B:", TEXT_B)
print(f"\nCommon sequence length (min of both): {common_length}")
print("Note: Both texts truncated to this length for clean superposition experiment")

# Decode to see tokens
tokens_A = [tokenizer.decode([t]) for t in input_ids_A[0].tolist()]
tokens_B = [tokenizer.decode([t]) for t in input_ids_B[0].tolist()]
print(f"\nTokens A ({len(tokens_A)}): {tokens_A}")
print(f"Tokens B ({len(tokens_B)}): {tokens_B}")
```

```
Text A: The capital of France is Paris, which is known for the Eiffel Tower
Text B: Machine learning algorithms can process large amounts of data efficiently

Common sequence length (min of both): 10
Note: Both texts truncated to this length for clean superposition experiment

Tokens A (10): ['The', ' capital', ' of', ' France', ' is', ' Paris', ',', ' which', ' is', ' known']
Tokens B (10): ['Machine', ' learning', ' algorithms', ' can', ' process', ' large', ' amounts', ' of', ' d
ata', ' efficiently']
```

In [10]:
```
# Run forward passes to extract Q, K, V
print("Running forward passes...")

# Clean pass A
result_A = extract_qkv_with_hooks(model, input_ids_A, attention_mask_A)
print("✓ Clean A done")

# Clean pass B
result_B = extract_qkv_with_hooks(model, input_ids_B, attention_mask_B)
print("✓ Clean B done")

# Mixed pass
mixed_embeds, embed_A, embed_B = create_mixed_embeddings(model, input_ids_A, input_ids_B)
result_mixed = forward_with_embeddings(model, mixed_embeds, attention_mask_mixed)
print("✓ Mixed A+B done")

print(f"\nExtracted {len(result_A.internals.queries)} layers of Q, K, V")
```

```
Running forward passes...
✓ Clean A done
✓ Clean B done
✓ Mixed A+B done

Extracted 32 layers of Q, K, V
```

In [11]:
```
def compute_sir_per_head(
    Q_mix: torch.Tensor,
    K_A: torch.Tensor,
    K_B: torch.Tensor,
    scale: float = None
) -> torch.Tensor:
    """
    Compute Signal-to-Interference Ratio for EACH attention head separately.

    SIR_head = ||Q_mix @ K_A^T||_F / ||Q_mix @ K_B^T||_F

    This avoids averaging across heads which would hide specialization.
```

```
    Args:
        Q_mix: Queries from mixed pass [batch, heads, seq, head_dim]
        K_A: Keys from clean A pass [batch, heads, seq, head_dim]
        K_B: Keys from clean B pass [batch, heads, seq, head_dim]
        scale: Scaling factor (1/sqrt(d_k)), computed if None

    Returns:
        SIR values per head [heads]
    """
    if scale is None:
        scale = 1.0 / np.sqrt(Q_mix.shape[-1])

    # Compute attention scores (before softmax)
    # Q @ K^T: [batch, heads, seq_q, seq_k]
    scores_A = torch.matmul(Q_mix.float(), K_A.float().transpose(-2, -1)) * scale
    scores_B = torch.matmul(Q_mix.float(), K_B.float().transpose(-2, -1)) * scale

    # Compute Frobenius norm PER HEAD
    # scores shape: [batch, heads, seq_q, seq_k]
    # We want norm over (seq_q, seq_k) for each head
    norm_signal_per_head = torch.norm(scores_A[0], dim=(-2, -1))  # [heads]
    norm_interference_per_head = torch.norm(scores_B[0], dim=(-2, -1))  # [heads]

    # SIR per head (avoid division by zero)
    sir_per_head = norm_signal_per_head / (norm_interference_per_head + 1e-10)

    return sir_per_head


# Collect SIR for EVERY head across ALL layers
all_sir_values = []  # Will be flat list of SIR values for all heads
sir_by_layer = []  # List of [heads] tensors for layer-wise analysis

print("Computing SIR per head for all layers...")

for layer_idx in range(num_layers):
    Q_mix = result_mixed.queries[layer_idx]
    K_A = result_A.internals.keys[layer_idx]
    K_B = result_B.internals.keys[layer_idx]

    sir_per_head = compute_sir_per_head(Q_mix, K_A, K_B)
    sir_by_layer.append(sir_per_head.numpy())
    all_sir_values.extend(sir_per_head.numpy().tolist())

all_sir_values = np.array(all_sir_values)
print(f"Collected SIR for {len(all_sir_values)} heads total ({num_layers} layers × {num_heads} heads)")
print(f"SIR range: [{all_sir_values.min():.4f}, {all_sir_values.max():.4f}]")
```

```
Computing SIR per head for all layers...
Collected SIR for 1024 heads total (32 layers × 32 heads)
SIR range: [0.6874, 1.4958]
```

In [12]:

```
# Visualize Log(SIR) distribution across ALL heads
# This is the KEY visualization - looking for bimodality!

log_sir = np.log(all_sir_values)  # Take log for better visualization

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Plot 1: MAIN - Histogram of Log(SIR) across all heads
ax = axes[0, 0]
ax.hist(log_sir, bins=50, density=True, alpha=0.7, color='steelblue', edgecolor='black')
# Add KDE
from scipy import stats as scipy_stats
kde = scipy_stats.gaussian_kde(log_sir)
x_range = np.linspace(log_sir.min() - 0.5, log_sir.max() + 0.5, 200)
ax.plot(x_range, kde(x_range), 'r-', linewidth=2, label='KDE')
ax.axvline(x=0, color='black', linestyle='--', linewidth=2, label='Log(SIR)=0 (SIR=1)')
ax.set_xlabel('Log(SIR) = Log(Signal_A / Signal_B)', fontsize=11)
ax.set_ylabel('Density')
ax.set_title('Distribution of Log(SIR) Across ALL Heads\n(Looking for bimodality: specialization)', fonts
ax.legend()
ax.grid(True, alpha=0.3)

# Add interpretation text
n_favor_A = (log_sir > 0.1).sum()
n_favor_B = (log_sir < -0.1).sum()
n_neutral = len(log_sir) - n_favor_A - n_favor_B
ax.text(0.02, 0.98, f'Heads favoring A (log>0.1): {n_favor_A}\n'
                    f'Heads favoring B (log<-0.1): {n_favor_B}\n'
                    f'Neutral heads: {n_neutral}',
        transform=ax.transAxes, fontsize=9, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

# Plot 2: Heatmap of SIR by layer and head
```

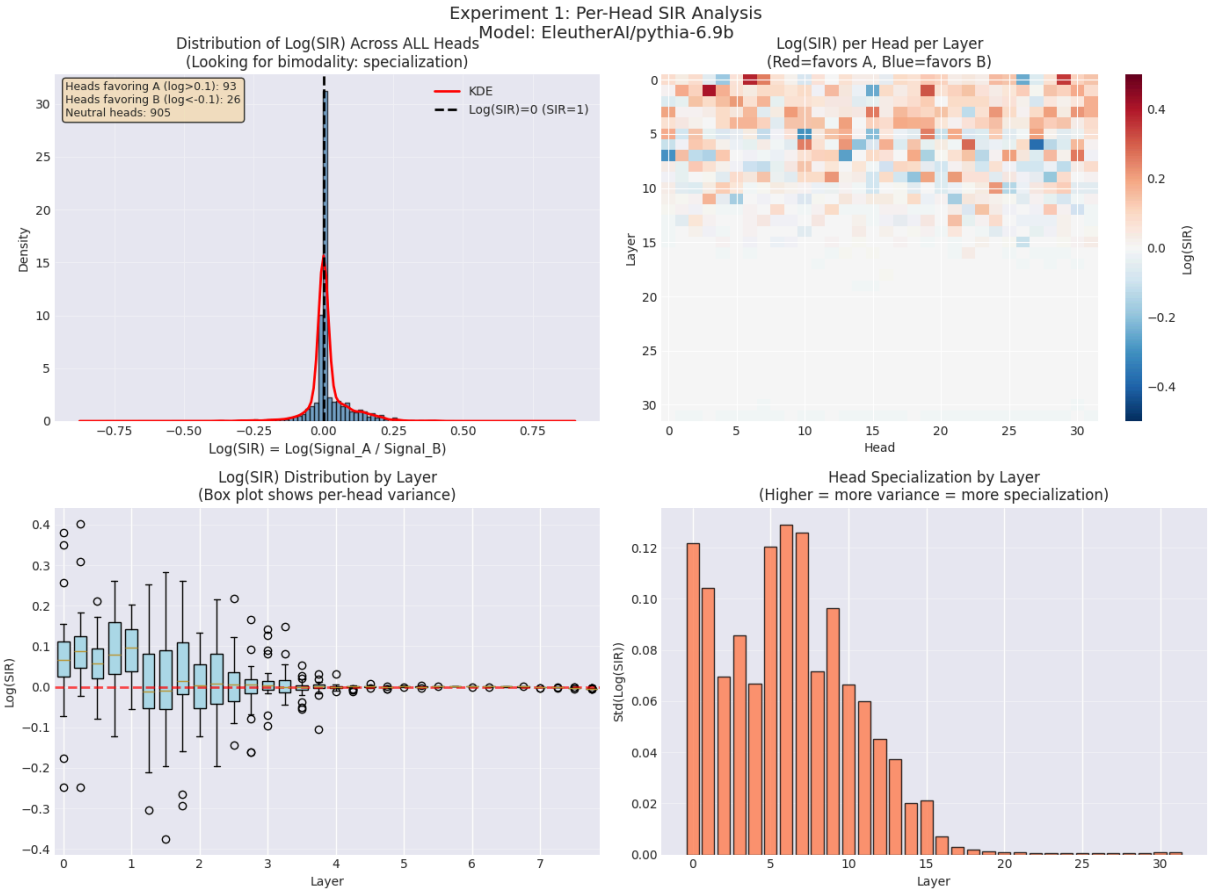
```
ax = axes[0, 1]
sir_matrix = np.array(sir_by_layer)  # [layers, heads]
im = ax.imshow(np.log(sir_matrix), aspect='auto', cmap='RdBu_r', vmin=-0.5, vmax=0.5)
ax.set_xlabel('Head')
ax.set_ylabel('Layer')
ax.set_title('Log(SIR) per Head per Layer\n(Red=favors A, Blue=favors B)')
plt.colorbar(im, ax=ax, label='Log(SIR)')

# Plot 3: Box plot of SIR by layer
ax = axes[1, 0]
bp = ax.boxplot([np.log(layer_sir) for layer_sir in sir_by_layer],
                positions=range(num_layers), widths=0.7, patch_artist=True)
for patch in bp['boxes']:
    patch.set_facecolor('lightblue')
ax.axhline(y=0, color='red', linestyle='--', linewidth=2, alpha=0.7)
ax.set_xlabel('Layer')
ax.set_ylabel('Log(SIR)')
ax.set_title('Log(SIR) Distribution by Layer\n(Box plot shows per-head variance)')
ax.set_xticks(range(0, num_layers, 4))
ax.grid(True, alpha=0.3, axis='y')

# Plot 4: Variance of Log(SIR) per layer (measure of specialization)
ax = axes[1, 1]
layer_variance = [np.var(np.log(layer_sir)) for layer_sir in sir_by_layer]
layer_std = [np.std(np.log(layer_sir)) for layer_sir in sir_by_layer]
ax.bar(range(num_layers), layer_std, color='coral', edgecolor='black', alpha=0.8)
ax.set_xlabel('Layer')
ax.set_ylabel('Std(Log(SIR))')
ax.set_title('Head Specialization by Layer\n(Higher = more variance = more specialization)')
ax.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.suptitle(f'Experiment 1: Per-Head SIR Analysis\nModel: {MODEL_NAME}', y=1.02, fontsize=14)
plt.savefig('exp1_sir_analysis.png', dpi=150, bbox_inches='tight')
plt.show()

# Summary statistics
print("\n" + "="*60)
print("SIR Summary Statistics (Per-Head Analysis)")
print("="*60)
print(f"Total heads analyzed: {len(all_sir_values)}")
print(f"\nLog(SIR) statistics:")
print(f"  Mean: {log_sir.mean():.4f} (SIR ≈ {np.exp(log_sir.mean()):.4f})")
print(f"  Std:  {log_sir.std():.4f}")
print(f"  Min:  {log_sir.min():.4f} (SIR = {np.exp(log_sir.min()):.4f})")
print(f"  Max:  {log_sir.max():.4f} (SIR = {np.exp(log_sir.max()):.4f})")
print(f"\nSpecialization analysis:")
print(f"  Heads favoring A (SIR > 1.1): {(all_sir_values > 1.1).sum()} ({100*(all_sir_values > 1.1).mean()
print(f"  Heads favoring B (SIR < 0.9): {(all_sir_values < 0.9).sum()} ({100*(all_sir_values < 0.9).mean()
print(f"  Neutral heads (0.9 ≤ SIR ≤ 1.1): {((all_sir_values >= 0.9) & (all_sir_values <= 1.1)).sum()}")
print(f"\nInterpretation:")
print(f"  • Bimodal distribution = heads specialize for A vs B")
print(f"  • Unimodal peak at 0 = no specialization, symmetric response")
print(f"  • High variance per layer = within-layer specialization")
```




```
============================================================
SIR Summary Statistics (Per-Head Analysis)
============================================================
Total heads analyzed: 1024

Log(SIR) statistics:
  Mean: 0.0146 (SIR ≈ 1.0147)
  Std:  0.0667
  Min:  -0.3748 (SIR = 0.6874)
  Max:  0.4027 (SIR = 1.4958)

Specialization analysis:
  Heads favoring A (SIR > 1.1): 98 (9.6%)
  Heads favoring B (SIR < 0.9): 26 (2.5%)
  Neutral heads (0.9 ≤ SIR ≤ 1.1): 900

Interpretation:
  • Bimodal distribution = heads specialize for A vs B
  • Unimodal peak at 0 = no specialization, symmetric response
  • High variance per layer = within-layer specialization
```

Experiment 2: "Ghost Context" Analysis (Value Leakage)

Question: When attention from stream A pulls values from stream B (because they share positions), what information "leaks" through?

Method:

1. Compute theoretical leakage: $\text{Leakage} = \text{AttnMap}_{A_clean} \times V_{B_clean}$
   - This represents: "What if we use A's attention pattern but B's values?"
2. Project leakage vector to vocabulary space using Logit Lens

3. Analyze: Is the leakage just noise, or does it contain coherent information from B?

In [13]:
```python
def compute_value_leakage(
    attn_weights_A: torch.Tensor,  # [batch, heads, seq, seq]
    values_B: torch.Tensor,  # [batch, heads, seq, head_dim]
) -> torch.Tensor:
    """
    Compute what "leaks" when attention pattern from A is applied to values from B.

    This simulates the "ghost context" phenomenon.

    Returns:
        Leakage vector [batch, heads, seq, head_dim]
    """
    # attn_weights_A @ values_B: [batch, heads, seq, head_dim]
    leakage = torch.matmul(attn_weights_A.float(), values_B.float())
    return leakage


def analyze_leakage_with_logit_lens(
    leakage: torch.Tensor,  # [batch, heads, seq, head_dim]
    lm_head_weights: torch.Tensor,
    layer_idx: int,
    position: int = -2,  # Analyze second-to-last position (before padding)
    top_k: int = 10
) -> Dict:
    """
    Analyze leakage vector using logit lens.

    For each head, project the leakage to vocabulary space and find top tokens.
    """
    batch_size, n_heads, seq_len, hdim = leakage.shape

    # Get leakage for the specified position
    # We need to aggregate across heads - let's look at each head separately
    results = {
        'layer': layer_idx,
        'position': position,
        'per_head_top_tokens': [],
        'aggregated_top_tokens': []
    }

    # Per-head analysis
    for head_idx in range(n_heads):
        head_leakage = leakage[0, head_idx, position, :]  # [head_dim]

        # We need to project head_dim to hidden_dim first
        # But head_dim is part of hidden_dim, so we can pad or use a different approach
        # For simplicity, let's skip per-head and use aggregated
        pass

    # Aggregated analysis: concatenate heads to get hidden_dim
    # [batch, heads, seq, head_dim] -> [batch, seq, heads * head_dim] = [batch, seq, hidden]
    leakage_combined = leakage[0, :, position, :].reshape(-1)  # [heads * head_dim]

    # Move lm_head_weights to same device as leakage
    lm_head_weights_device = lm_head_weights.to(leakage_combined.device)

    # Project to vocabulary
    logits = leakage_combined.float() @ lm_head_weights_device.float().T  # [vocab]
    top_values, top_indices = torch.topk(logits, top_k)

    for val, idx in zip(top_values, top_indices):
        token_str = tokenizer.decode([idx.item()])
        results['aggregated_top_tokens'].append({
            'token': token_str,
            'token_id': idx.item(),
            'logit': val.item()
        })

    return results
```

In [14]:
```python
# Get LM head weights
lm_head_weights = get_lm_head_weights(model)

# Sequence length is the same for both (common_length, no padding)
actual_len = common_length
print(f"Sequence length (both texts): {actual_len}")

# Analyze leakage at different layers
```

```python
leakage_analysis = []
layers_to_analyze = [0, num_layers // 4, num_layers // 2, 3 * num_layers // 4, num_layers - 1]

print("\n" + "="*80)
print("VALUE LEAKAGE ANALYSIS (Ghost Context)")
print("="*80)
print(f"Question: What tokens appear when we apply A's attention to B's values?")
print(f"Text B tokens (source of leakage): {tokens_B}")
print("="*80)

for layer_idx in layers_to_analyze:
    print(f"\n--- Layer {layer_idx} ---")

    # Get attention weights from clean A run and values from clean B run
    attn_A = result_A.internals.attention_weights[layer_idx]  # [batch, heads, seq, seq]
    values_B = result_B.internals.values[layer_idx]  # [batch, heads, seq, head_dim]

    # Compute leakage
    leakage = compute_value_leakage(attn_A, values_B)

    # Analyze at the last token position
    analysis_pos = actual_len - 1

    analysis = analyze_leakage_with_logit_lens(
        leakage, lm_head_weights, layer_idx, position=analysis_pos, top_k=10
    )
    leakage_analysis.append(analysis)

    print(f"Position analyzed: {analysis_pos}")
    print("Top tokens in leakage projection:")
    for i, tok_info in enumerate(analysis['aggregated_top_tokens'][:5]):
        token_in_B = tok_info['token'] in tokens_B
        marker = " ← FROM TEXT B!" if token_in_B else ""
        print(f"  {i+1}. '{tok_info['token']}' (logit: {tok_info['logit']:.2f}){marker}")
```

```
Sequence length (both texts): 10

================================================================================
VALUE LEAKAGE ANALYSIS (Ghost Context)
================================================================================
Question: What tokens appear when we apply A's attention to B's values?
Text B tokens (source of leakage): ['Machine', ' learning', ' algorithms', ' can', ' process', ' large', '
amounts', ' of', ' data', ' efficiently']
================================================================================

--- Layer 0 ---
Position analyzed: 9
Top tokens in leakage projection:
  1. 'ode' (logit: 0.87)
  2. 'ment' (logit: 0.87)
  3. 'iet' (logit: 0.86)
  4. ' lay' (logit: 0.86)
  5. 'tec' (logit: 0.86)

--- Layer 8 ---
Position analyzed: 9
Top tokens in leakage projection:
  1. 'kin' (logit: 0.93)
  2. 'iya' (logit: 0.87)
  3. 'band' (logit: 0.81)
  4. 'ouin' (logit: 0.81)
  5. 'Band' (logit: 0.77)

--- Layer 16 ---
Position analyzed: 9
Top tokens in leakage projection:
  1. 'SEE' (logit: 1.85)
  2. 'Advertisement' (logit: 1.65)
  3. 'ynn' (logit: 1.58)
  4. 'enez' (logit: 1.54)
  5. ' describ' (logit: 1.37)

--- Layer 24 ---
Position analyzed: 9
Top tokens in leakage projection:
  1. ' msgid' (logit: 1.41)
  2. 'eria' (logit: 1.35)
  3. 'otta' (logit: 1.18)
  4. 'ois' (logit: 1.12)
  5. ' them' (logit: 1.11)

--- Layer 31 ---
Position analyzed: 9
Top tokens in leakage projection:
  1. 'olas' (logit: 1.58)
  2. 'deal' (logit: 1.57)
  3. ' Fro' (logit: 1.51)
  4. 'beck' (logit: 1.48)
  5. 'arity' (logit: 1.45)
```

In [15]:

```python
# Visualize LOGIT IMPACT instead of norms
# Key insight: A long vector pointing nowhere is useless; a short vector pointing at the right token is po

def compute_logit_impact(output_vector: torch.Tensor, lm_head_weights: torch.Tensor) -> Dict[str, float]:
    """
    Compute the effective impact of a vector on logits.

    Metric: EffectiveLogitStrength = Max(Logits) - Mean(Logits)
    This measures how much the vector "spikes" specific tokens vs noise.

    Args:
        output_vector: [batch, seq, hidden] or [batch, heads, seq, head_dim]
        lm_head_weights: [vocab_size, hidden_dim]

    Returns:
        Dictionary with logit impact metrics
    """
    # Handle different input shapes
    if output_vector.dim() == 4:
        # [batch, heads, seq, head_dim] -> [batch, seq, hidden]
        batch, heads, seq, hdim = output_vector.shape
        output_vector = output_vector.transpose(1, 2).reshape(batch, seq, heads * hdim)

    # Project to vocabulary space
    # [batch, seq, hidden] @ [hidden, vocab] = [batch, seq, vocab]
    logits = output_vector.float() @ lm_head_weights.float().T

    # Compute metrics per position, then average
    max_logits = logits.max(dim=-1).values  # [batch, seq]
    mean_logits = logits.mean(dim=-1)  # [batch, seq]
```

```
    # Effective strength: how much above mean is the max?
    effective_strength = (max_logits - mean_logits).mean().item()

    # Also compute std of logits (another measure of "peakiness")
    logit_std = logits.std(dim=-1).mean().item()

    # Top-k concentration: what fraction of probability mass is in top 10 tokens?
    probs = F.softmax(logits, dim=-1)
    top10_prob = probs.topk(10, dim=-1).values.sum(dim=-1).mean().item()

    return {
        'effective_strength': effective_strength,
        'logit_std': logit_std,
        'top10_concentration': top10_prob,
        'max_logit': max_logits.mean().item(),
        'mean_logit': mean_logits.mean().item()
    }


# Compute logit impact for Signal vs Leakage at each layer
signal_impacts = []
leakage_impacts = []

print("Computing logit impact for Signal vs Leakage...")

for layer_idx in tqdm(range(num_layers)):
    attn_A = result_A.internals.attention_weights[layer_idx]
    values_A = result_A.internals.values[layer_idx]
    values_B = result_B.internals.values[layer_idx]

    # Signal: A's attention × A's values
    signal_output = torch.matmul(attn_A.float(), values_A.float())
    # Leakage: A's attention × B's values (ghost context)
    leakage_output = torch.matmul(attn_A.float(), values_B.float())

    sig_impact = compute_logit_impact(signal_output, lm_head_weights.cpu())
    leak_impact = compute_logit_impact(leakage_output, lm_head_weights.cpu())

    signal_impacts.append(sig_impact)
    leakage_impacts.append(leak_impact)

# Plot
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Plot 1: Effective Logit Strength
ax = axes[0, 0]
ax.plot(range(num_layers), [s['effective_strength'] for s in signal_impacts],
        'b-o', label='Signal (AttnA × ValuesA)', linewidth=2)
ax.plot(range(num_layers), [l['effective_strength'] for l in leakage_impacts],
        'r-o', label='Leakage (AttnA × ValuesB)', linewidth=2)
ax.set_xlabel('Layer')
ax.set_ylabel('Max(Logits) - Mean(Logits)')
ax.set_title('Effective Logit Strength\n(How much the vector "spikes" specific tokens)')
ax.legend()
ax.grid(True, alpha=0.3)

# Plot 2: Signal-to-Leakage Ratio (in logit impact)
ax = axes[0, 1]
impact_ratios = [s['effective_strength'] / (l['effective_strength'] + 1e-10)
                 for s, l in zip(signal_impacts, leakage_impacts)]
ax.plot(range(num_layers), impact_ratios, 'g-o', linewidth=2)
ax.axhline(y=1.0, color='r', linestyle='--', alpha=0.5, label='Ratio=1')
ax.set_xlabel('Layer')
ax.set_ylabel('Signal / Leakage (Logit Impact)')
ax.set_title('Signal-to-Leakage Ratio in Logit Space\n(>1 means signal dominates prediction)')
ax.legend()
ax.grid(True, alpha=0.3)

# Plot 3: Top-10 Concentration
ax = axes[1, 0]
ax.plot(range(num_layers), [s['top10_concentration'] for s in signal_impacts],
        'b-o', label='Signal', linewidth=2)
ax.plot(range(num_layers), [l['top10_concentration'] for l in leakage_impacts],
        'r-o', label='Leakage', linewidth=2)
ax.set_xlabel('Layer')
ax.set_ylabel('Prob mass in top 10 tokens')
ax.set_title('Token Concentration\n(How "focused" is the prediction?)')
ax.legend()
ax.grid(True, alpha=0.3)

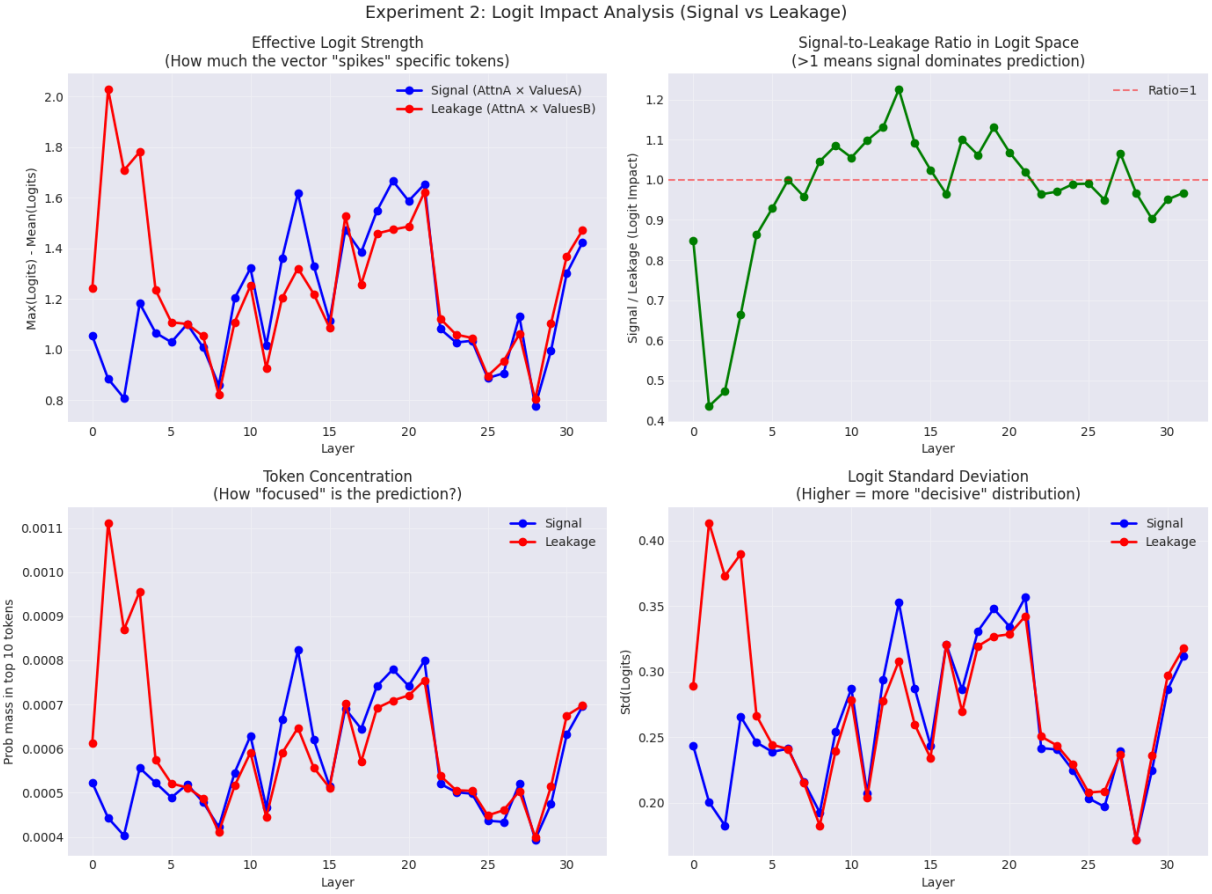
# Plot 4: Logit Std (peakiness)
ax = axes[1, 1]
ax.plot(range(num_layers), [s['logit_std'] for s in signal_impacts],
        'b-o', label='Signal', linewidth=2)
```

```python
ax.plot(range(num_layers), [l['logit_std'] for l in leakage_impacts],
        'r-o', label='Leakage', linewidth=2)
ax.set_xlabel('Layer')
ax.set_ylabel('Std(Logits)')
ax.set_title('Logit Standard Deviation\n(Higher = more "decisive" distribution)')
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.suptitle('Experiment 2: Logit Impact Analysis (Signal vs Leakage)', y=1.02, fontsize=14)
plt.savefig('exp2_value_leakage.png', dpi=150, bbox_inches='tight')
plt.show()

# Summary
print("\n" + "="*60)
print("Logit Impact Summary")
print("="*60)
print(f"Mean Signal/Leakage ratio (logit impact): {np.mean(impact_ratios):.3f}")
print(f"Signal effective strength (last layer): {signal_impacts[-1]['effective_strength']:.3f}")
print(f"Leakage effective strength (last layer): {leakage_impacts[-1]['effective_strength']:.3f}")
print(f"\nInterpretation:")
print(f"  • Ratio > 1: Signal points more strongly at specific tokens than leakage")
print(f"  • This is what matters for prediction, not just vector norms!")
print(f"  • High top-10 concentration = coherent prediction")
print(f"  • Low leakage concentration = leakage is diffuse noise")
```

```
Computing logit impact for Signal vs Leakage...
  0%|          | 0/32 [00:00<?, ?it/s]
```




```
============================================================
Logit Impact Summary
============================================================
Mean Signal/Leakage ratio (logit impact): 0.968
Signal effective strength (last layer): 1.425
Leakage effective strength (last layer): 1.473

Interpretation:
  • Ratio > 1: Signal points more strongly at specific tokens than leakage
  • This is what matters for prediction, not just vector norms!
  • High top-10 concentration = coherent prediction
  • Low leakage concentration = leakage is diffuse noise
```

Experiment 3: Attention Pattern Overlap vs Quality

Hypothesis: If both texts A and B attend to the same positions, interference should be worse.

Method:

1. Compute attention pattern similarity between A and B (cosine similarity, IoU)
2. For multiple text pairs, measure:
   - Attention overlap
   - Rank of correct token in mixed prediction
3. Check correlation: high overlap → worse quality?

In [16]:

```python
def compute_off_diagonal_overlap(
    attn_A: torch.Tensor,  # [batch, heads, seq, seq]
    attn_B: torch.Tensor,
    zero_diagonal: bool = True,
    zero_first_column: bool = True,
    method: str = 'cosine'
) -> Dict[str, torch.Tensor]:
    """
    Compute overlap between attention patterns from A and B,
    EXCLUDING trivial components (diagonal and first token / attention sink).

    Why this matters:
    - Diagonal attention (token attending to itself) is trivial and universal
    - First token attention (sink) is a known artifact
    - These inflate overlap artificially, hiding real cross-talk
    - We care about INDUCTION patterns (looking back into history)

    Args:
        attn_A, attn_B: Attention weights [batch, heads, seq, seq]
        zero_diagonal: If True, set diagonal attention to 0 before comparison
        zero_first_column: If True, set attention to first token to 0
        method: 'cosine' or 'iou'

    Returns:
        Dictionary with overlap metrics
    """
    batch, n_heads, seq_q, seq_k = attn_A.shape

    # Clone to avoid modifying original
    attn_A_clean = attn_A.clone().float()
    attn_B_clean = attn_B.clone().float()

    if zero_diagonal:
        # Zero out diagonal (token attending to itself)
        # For causal attention, diagonal exists where i <= j
        for i in range(min(seq_q, seq_k)):
            attn_A_clean[:, :, i, i] = 0
            attn_B_clean[:, :, i, i] = 0

    if zero_first_column:
        # Zero out attention to first token (attention sink)
        attn_A_clean[:, :, :, 0] = 0
        attn_B_clean[:, :, :, 0] = 0

    # Re-normalize after zeroing (so we compare proper distributions)
    attn_A_clean = attn_A_clean / (attn_A_clean.sum(dim=-1, keepdim=True) + 1e-10)
    attn_B_clean = attn_B_clean / (attn_B_clean.sum(dim=-1, keepdim=True) + 1e-10)

    if method == 'cosine':
        # Normalize for cosine similarity
        attn_A_norm = F.normalize(attn_A_clean, p=2, dim=-1)
        attn_B_norm = F.normalize(attn_B_clean, p=2, dim=-1)

        # Cosine similarity: sum(a*b)
        cosine_sim = (attn_A_norm * attn_B_norm).sum(dim=-1)  # [batch, heads, seq_q]

        return {
```

```
            'per_position': cosine_sim,
            'mean_per_head': cosine_sim.mean(dim=-1),  # [batch, heads]
            'overall_mean': cosine_sim.mean().item()
        }

    elif method == 'iou':
        # Intersection over Union for top-k attended positions (excluding diagonal/first)
        top_k = 5

        _, top_A = torch.topk(attn_A_clean, top_k, dim=-1)
        _, top_B = torch.topk(attn_B_clean, top_k, dim=-1)

        iou_scores = []
        for b in range(batch):
            for h in range(n_heads):
                for q in range(seq_q):
                    set_A = set(top_A[b, h, q].tolist())
                    set_B = set(top_B[b, h, q].tolist())
                    intersection = len(set_A & set_B)
                    union = len(set_A | set_B)
                    iou_scores.append(intersection / union if union > 0 else 0)

        return {
            'iou_scores': iou_scores,
            'mean_iou': np.mean(iou_scores)
        }

    else:
        raise ValueError(f"Unknown method: {method}")


# Also keep the original for comparison
def compute_attention_overlap_original(
    attn_A: torch.Tensor,
    attn_B: torch.Tensor,
) -> float:
    """Original overlap (including diagonal) for comparison."""
    attn_A_norm = F.normalize(attn_A.float(), p=2, dim=-1)
    attn_B_norm = F.normalize(attn_B.float(), p=2, dim=-1)
    return (attn_A_norm * attn_B_norm).sum(dim=-1).mean().item()


def compute_prediction_rank(
    logits: torch.Tensor,  # [batch, seq, vocab]
    target_ids: torch.Tensor,  # [batch, seq]
    position: int = -2
) -> List[int]:
    """
    Compute rank of target token in prediction distribution.
    Rank 1 = correct token is top prediction.
    """
    batch = logits.shape[0]
    ranks = []

    for b in range(batch):
        pos_logits = logits[b, position, :]
        target_id = target_ids[b, position + 1].item()  # Next token prediction

        # Sort logits descending
        sorted_indices = torch.argsort(pos_logits, descending=True)
        rank = (sorted_indices == target_id).nonzero(as_tuple=True)[0].item() + 1
        ranks.append(rank)

    return ranks
```

In [17]:

```
# Compute attention overlap for each layer
# CRITICAL: Compare WITH and WITHOUT diagonal/first-token zeroing

overlap_with_diagonal = []  # Original (inflated)
overlap_off_diagonal = []   # Corrected (meaningful)

print("Computing attention overlap (with and without diagonal/sink)...")

for layer_idx in range(num_layers):
    attn_A = result_A.internals.attention_weights[layer_idx]
    attn_B = result_B.internals.attention_weights[layer_idx]

    # Original overlap (includes trivial diagonal)
    original = compute_attention_overlap_original(attn_A, attn_B)
    overlap_with_diagonal.append(original)

    # Corrected overlap (excludes diagonal and first token)
    corrected = compute_off_diagonal_overlap(
        attn_A, attn_B,
```

```
        zero_diagonal=True,
        zero_first_column=True,
        method='cosine'
    )
    corrected['layer'] = layer_idx
    overlap_off_diagonal.append(corrected)

# Visualize overlap across layers
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Plot 1: Comparison of original vs corrected overlap
ax = axes[0, 0]
layer_overlaps_original = overlap_with_diagonal
layer_overlaps_corrected = [r['overall_mean'] for r in overlap_off_diagonal]

ax.plot(range(num_layers), layer_overlaps_original, 'r-o', linewidth=2,
        label='Original (with diagonal)', alpha=0.7)
ax.plot(range(num_layers), layer_overlaps_corrected, 'b-o', linewidth=2,
        label='Off-diagonal only', alpha=0.7)
ax.fill_between(range(num_layers), layer_overlaps_original, layer_overlaps_corrected,
                alpha=0.3, color='gray', label='Inflation from diagonal')
ax.set_xlabel('Layer')
ax.set_ylabel('Mean Cosine Similarity')
ax.set_title('Attention Overlap: Original vs Off-Diagonal\n(Gray area = trivial diagonal inflation)')
ax.axhline(y=0.0, color='gray', linestyle='--', alpha=0.5)
ax.legend()
ax.grid(True, alpha=0.3)
ax.set_ylim(-0.1, 1.0)

# Plot 2: Off-diagonal overlap only (the meaningful metric)
ax = axes[0, 1]
ax.plot(range(num_layers), layer_overlaps_corrected, 'b-o', linewidth=2)
ax.set_xlabel('Layer')
ax.set_ylabel('Off-Diagonal Cosine Similarity')
ax.set_title('TRUE Attention Overlap (History/Induction only)\nExcludes diagonal & first token')
ax.axhline(y=0.0, color='gray', linestyle='--', alpha=0.5)
ax.grid(True, alpha=0.3)
ax.set_ylim(-0.1, 1.0)

# Add annotation about inflation
mean_inflation = np.mean(layer_overlaps_original) - np.mean(layer_overlaps_corrected)
ax.text(0.02, 0.98, f'Mean inflation removed: {mean_inflation:.3f}\n'
                    f'Original mean: {np.mean(layer_overlaps_original):.3f}\n'
                    f'Corrected mean: {np.mean(layer_overlaps_corrected):.3f}',
        transform=ax.transAxes, fontsize=9, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

# Plot 3: Heatmap of corrected overlap per head (for a few layers)
ax = axes[1, 0]
sample_layers = [0, num_layers // 4, num_layers // 2, 3 * num_layers // 4, num_layers - 1]
head_overlaps = np.zeros((len(sample_layers), num_heads))

for i, layer_idx in enumerate(sample_layers):
    head_overlaps[i, :] = overlap_off_diagonal[layer_idx]['mean_per_head'][0].numpy()

im = ax.imshow(head_overlaps, aspect='auto', cmap='RdYlBu_r', vmin=-0.2, vmax=0.8)
ax.set_xlabel('Head')
ax.set_ylabel('Layer')
ax.set_yticks(range(len(sample_layers)))
ax.set_yticklabels([f'Layer {l}' for l in sample_layers])
ax.set_title('Off-Diagonal Overlap per Head\n(Blue=different patterns, Red=similar patterns)')
plt.colorbar(im, ax=ax, label='Cosine Similarity')

# Plot 4: Distribution of per-head overlaps
ax = axes[1, 1]
all_head_overlaps = []
for layer_idx in range(num_layers):
    head_vals = overlap_off_diagonal[layer_idx]['mean_per_head'][0].numpy()
    all_head_overlaps.extend(head_vals)

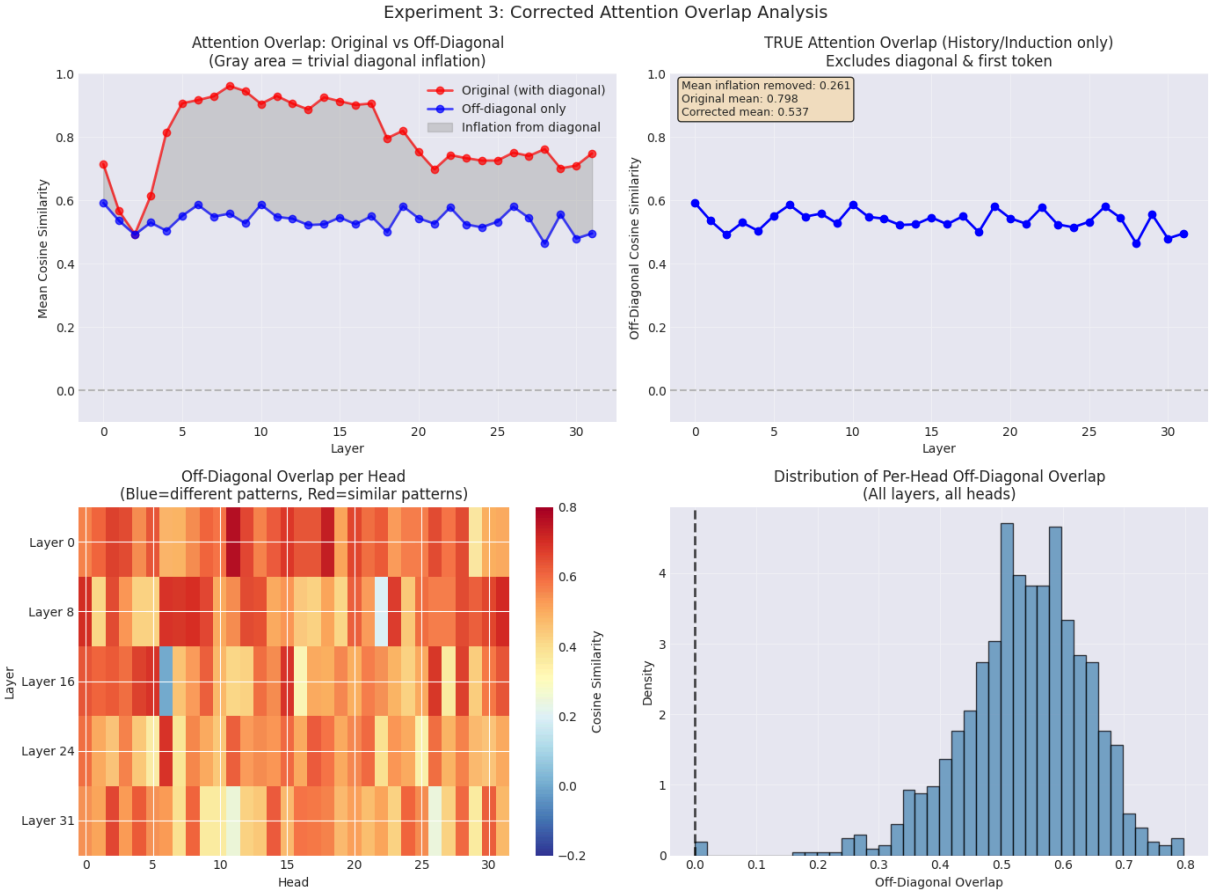
ax.hist(all_head_overlaps, bins=40, density=True, alpha=0.7, color='steelblue', edgecolor='black')
ax.axvline(x=0, color='black', linestyle='--', linewidth=2, alpha=0.7)
ax.set_xlabel('Off-Diagonal Overlap')
ax.set_ylabel('Density')
ax.set_title('Distribution of Per-Head Off-Diagonal Overlap\n(All layers, all heads)')
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.suptitle('Experiment 3: Corrected Attention Overlap Analysis', y=1.02, fontsize=14)
plt.savefig('exp3_attention_overlap.png', dpi=150, bbox_inches='tight')
plt.show()

print("\n" + "="*60)
print("Attention Overlap Summary")
print("="*60)
```

```
print(f"ORIGINAL (with diagonal + first token):")
print(f"  Mean overlap: {np.mean(layer_overlaps_original):.4f}")
print(f"  This is MISLEADING - inflated by trivial patterns!")
print(f"\nCORRECTED (off-diagonal only):")
print(f"  Mean overlap: {np.mean(layer_overlaps_corrected):.4f}")
print(f"  Max at layer {np.argmax(layer_overlaps_corrected)}: {np.max(layer_overlaps_corrected):.4f}")
print(f"  Min at layer {np.argmin(layer_overlaps_corrected)}: {np.min(layer_overlaps_corrected):.4f}")
print(f"\nInflation from diagonal: {mean_inflation:.4f} ({100*mean_inflation/np.mean(layer_overlaps_origi
print(f"\nInterpretation:")
print(f"  • Low off-diagonal overlap = A and B look at different history")
print(f"  • High overlap = potential for interference in cross-attention")
```

```
Computing attention overlap (with and without diagonal/sink)...
```




```
============================================================
Attention Overlap Summary
============================================================
ORIGINAL (with diagonal + first token):
  Mean overlap: 0.7979
  This is MISLEADING - inflated by trivial patterns!

CORRECTED (off-diagonal only):
  Mean overlap: 0.5368
  Max at layer 0: 0.5917
  Min at layer 28: 0.4632

Inflation from diagonal: 0.2611 (32.7% of original)

Interpretation:
  • Low off-diagonal overlap = A and B look at different history
  • High overlap = potential for interference in cross-attention
```

Statistical Analysis: Overlap vs Quality Correlation

Now we run the analysis on multiple text pairs to see if higher overlap correlates with worse prediction quality.

In [18]:

```
# Sample text pairs for statistical analysis
TEXT_PAIRS = [
    # Similar topic pairs (might have higher overlap)
    ("The president announced new economic policies", "The government proposed budget reforms"),
```

```
    ("Python is a programming language", "JavaScript runs in web browsers"),
    ("The cat sat on the mat", "The dog slept on the couch"),

    # Dissimilar topic pairs (should have lower overlap)
    ("Quantum physics explores subatomic particles", "Chocolate cake is delicious"),
    ("The stock market crashed yesterday", "Beautiful flowers bloom in spring"),
    ("Neural networks learn from data", "The ancient Romans built roads"),

    # Technical vs casual
    ("Machine learning models require training", "My favorite movie is Star Wars"),
    ("HTTP requests use TCP connections", "The weather is nice today"),

    # Numbers vs text
    ("The temperature is twenty degrees", "Mathematical equations solve problems"),
    ("First second third fourth fifth", "Alpha beta gamma delta epsilon"),
]

def analyze_text_pair(text_a: str, text_b: str, analysis_layer: int = None):
    """
    Analyze a single text pair and return overlap and quality metrics.
    Uses minimum length truncation for clean experiment.
    NOW USES OFF-DIAGONAL OVERLAP (corrected metric).
    """
    if analysis_layer is None:
        analysis_layer = num_layers // 2

    # Tokenize with min-length truncation for clean superposition
    ids_a, mask_a, ids_b, mask_b, pair_len = tokenize_pair_to_min_length(text_a, text_b)

    if pair_len < 3:
        return None  # Too short for meaningful analysis

    # Forward passes
    with torch.no_grad():
        # Clean passes
        out_a = model(ids_a, attention_mask=mask_a, output_attentions=True, output_hidden_states=True)
        out_b = model(ids_b, attention_mask=mask_b, output_attentions=True, output_hidden_states=True)

        # Mixed pass
        mixed_emb, _, _ = create_mixed_embeddings(model, ids_a, ids_b)
        out_mixed = model(inputs_embeds=mixed_emb, attention_mask=mask_a, output_attentions=True)

    # Compute CORRECTED overlap (off-diagonal only!)
    attn_a = out_a.attentions[analysis_layer]
    attn_b = out_b.attentions[analysis_layer]

    # Original overlap for comparison
    overlap_original = compute_attention_overlap_original(attn_a.cpu(), attn_b.cpu())

    # Corrected overlap (the meaningful one)
    overlap_corrected = compute_off_diagonal_overlap(
        attn_a.cpu(), attn_b.cpu(),
        zero_diagonal=True,
        zero_first_column=True,
        method='cosine'
    )['overall_mean']

    # Compute prediction quality (rank of correct token)
    # Use second-to-last position for next-token prediction
    analysis_pos = pair_len - 2

    if analysis_pos < 1:
        return None

    # Rank for A and B
    logits_mixed = out_mixed.logits.cpu()
    target_a = ids_a.cpu()[0, analysis_pos + 1].item()
    target_b = ids_b.cpu()[0, analysis_pos + 1].item()

    pos_logits = logits_mixed[0, analysis_pos, :]
    sorted_indices = torch.argsort(pos_logits, descending=True)

    rank_a = (sorted_indices == target_a).nonzero(as_tuple=True)[0].item() + 1
    rank_b = (sorted_indices == target_b).nonzero(as_tuple=True)[0].item() + 1

    return {
        'text_a': text_a[:50] + '...' if len(text_a) > 50 else text_a,
        'text_b': text_b[:50] + '...' if len(text_b) > 50 else text_b,
        'common_length': pair_len,
        'overlap_original': overlap_original,
        'overlap': overlap_corrected,  # This is now the corrected metric
        'rank_a': rank_a,
        'rank_b': rank_b,
        'mean_rank': (rank_a + rank_b) / 2
    }
```

In [19]:
```
# Run analysis on all text pairs
print("Analyzing text pairs...")
pair_results = []

for text_a, text_b in tqdm(TEXT_PAIRS):
    result = analyze_text_pair(text_a, text_b)
    if result:
        pair_results.append(result)
        print(f"  Overlap: {result['overlap']:.4f}, Rank A: {result['rank_a']}, Rank B: {result['rank_b']}

print(f"\nAnalyzed {len(pair_results)} text pairs")
```

```
Analyzing text pairs...
  0%|          | 0/10 [00:00<?, ?it/s]
  Overlap: 0.5301, Rank A: 24, Rank B: 13
  Overlap: 0.5041, Rank A: 1, Rank B: 59
  Overlap: 0.5768, Rank A: 13, Rank B: 4
  Overlap: 0.4808, Rank A: 14066, Rank B: 392
  Overlap: 0.5207, Rank A: 22, Rank B: 1
  Overlap: 0.5195, Rank A: 5, Rank B: 146
  Overlap: 0.5224, Rank A: 257, Rank B: 222
  Overlap: 0.5144, Rank A: 14370, Rank B: 25
  Overlap: 0.5188, Rank A: 80, Rank B: 86
  Overlap: 0.5022, Rank A: 1, Rank B: 53

Analyzed 10 text pairs
```

In [20]:
```
# Visualize correlation between overlap and quality
# NOW COMPARING BOTH ORIGINAL AND CORRECTED OVERLAP

if len(pair_results) > 2:
    overlaps_original = [r['overlap_original'] for r in pair_results]
    overlaps_corrected = [r['overlap'] for r in pair_results]  # This is now off-diagonal
    mean_ranks = [r['mean_rank'] for r in pair_results]

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    from scipy import stats

    # Plot 1: Scatter plot with CORRECTED overlap (the meaningful one)
    ax = axes[0, 0]
    ax.scatter(overlaps_corrected, mean_ranks, s=100, alpha=0.7, c='blue', edgecolors='black')
    for i, r in enumerate(pair_results):
        ax.annotate(f"{i+1}", (overlaps_corrected[i], mean_ranks[i]), fontsize=8, ha='center', va='bottom
    ax.set_xlabel('Off-Diagonal Overlap (Corrected)')
    ax.set_ylabel('Mean Rank of Correct Token')
    ax.set_title('CORRECTED Overlap vs Prediction Quality\n(Excludes diagonal & first token)')
    ax.grid(True, alpha=0.3)

    # Correlation for corrected
    corr_corrected, p_corrected = stats.pearsonr(overlaps_corrected, mean_ranks)
    ax.text(0.05, 0.95, f'Pearson r = {corr_corrected:.3f}\np = {p_corrected:.3f}',
            transform=ax.transAxes, fontsize=10, verticalalignment='top',
            bbox=dict(boxstyle='round', facecolor='lightgreen' if p_corrected < 0.1 else 'wheat', alpha=0

    # Plot 2: Scatter plot with ORIGINAL overlap (for comparison)
    ax = axes[0, 1]
    ax.scatter(overlaps_original, mean_ranks, s=100, alpha=0.7, c='red', edgecolors='black')
    for i, r in enumerate(pair_results):
        ax.annotate(f"{i+1}", (overlaps_original[i], mean_ranks[i]), fontsize=8, ha='center', va='bottom'
    ax.set_xlabel('Original Overlap (with diagonal)')
    ax.set_ylabel('Mean Rank of Correct Token')
    ax.set_title('ORIGINAL Overlap vs Quality\n(Inflated by diagonal - for comparison)')
    ax.grid(True, alpha=0.3)

    # Correlation for original
    corr_original, p_original = stats.pearsonr(overlaps_original, mean_ranks)
    ax.text(0.05, 0.95, f'Pearson r = {corr_original:.3f}\np = {p_original:.3f}',
            transform=ax.transAxes, fontsize=10, verticalalignment='top',
            bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

    # Plot 3: Bar chart of ranks
    ax = axes[1, 0]
    x = np.arange(len(pair_results))
    width = 0.35

    ax.bar(x - width/2, [r['rank_a'] for r in pair_results], width, label='Rank A', alpha=0.7, color='ste
    ax.bar(x + width/2, [r['rank_b'] for r in pair_results], width, label='Rank B', alpha=0.7, color='cor
    ax.set_xlabel('Text Pair')
    ax.set_ylabel('Rank of Correct Token')
    ax.set_title('Prediction Ranks per Text Pair')
    ax.legend()
    ax.set_xticks(x)
```

```
    ax.set_xticklabels([f'{i+1}' for i in range(len(pair_results))])
    ax.set_yscale('log')
    ax.grid(True, alpha=0.3, axis='y')

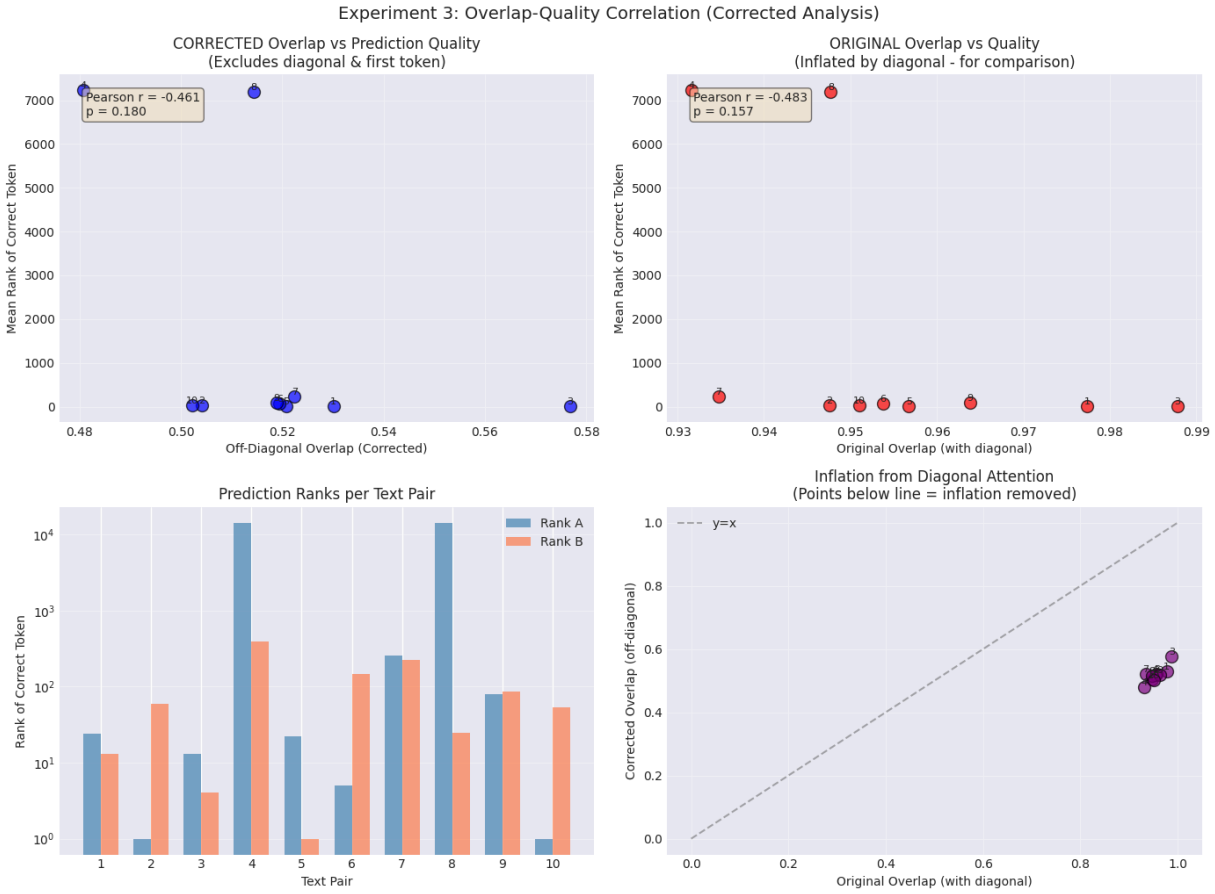
    # Plot 4: Original vs Corrected overlap comparison
    ax = axes[1, 1]
    ax.scatter(overlaps_original, overlaps_corrected, s=100, alpha=0.7, c='purple', edgecolors='black')
    for i in range(len(pair_results)):
        ax.annotate(f"{i+1}", (overlaps_original[i], overlaps_corrected[i]), fontsize=8, ha='center', va=
    ax.plot([0, 1], [0, 1], 'k--', alpha=0.3, label='y=x')
    ax.set_xlabel('Original Overlap (with diagonal)')
    ax.set_ylabel('Corrected Overlap (off-diagonal)')
    ax.set_title('Inflation from Diagonal Attention\n(Points below line = inflation removed)')
    ax.legend()
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.suptitle('Experiment 3: Overlap-Quality Correlation (Corrected Analysis)', y=1.02, fontsize=14)
    plt.savefig('exp3_overlap_quality.png', dpi=150, bbox_inches='tight')
    plt.show()

    print("\n" + "="*60)
    print("Correlation Analysis")
    print("="*60)
    print(f"\nCORRECTED Overlap (off-diagonal):")
    print(f"  Pearson correlation: {corr_corrected:.4f}")
    print(f"  P-value: {p_corrected:.4f}")

    print(f"\nORIGINAL Overlap (with diagonal):")
    print(f"  Pearson correlation: {corr_original:.4f}")
    print(f"  P-value: {p_original:.4f}")

    print(f"\nInterpretation:")
    if p_corrected < 0.05:
        print(f"  ✓ SIGNIFICANT correlation with corrected overlap (p < 0.05)")
        if corr_corrected > 0:
            print(f"  → Higher off-diagonal overlap → Worse quality (higher rank)")
            print(f"  → This supports interference hypothesis!")
        else:
            print(f"  → Higher off-diagonal overlap → Better quality (lower rank)")
    else:
        print(f"  ✗ No significant correlation with corrected overlap")
        if p_original < 0.05 and p_corrected >= 0.05:
            print(f"  → Original correlation was spurious (driven by diagonal)")
else:
    print("Not enough data points for correlation analysis")
```




```
============================================================
Correlation Analysis
============================================================

CORRECTED Overlap (off-diagonal):
  Pearson correlation: -0.4611
  P-value: 0.1799

ORIGINAL Overlap (with diagonal):
  Pearson correlation: -0.4830
  P-value: 0.1574

Interpretation:
  ✗ No significant correlation with corrected overlap
```

Summary and Conclusions

Key Findings from Mechanistic Analysis

In [21]:

```python
# Create summary visualization with CORRECTED metrics
fig = plt.figure(figsize=(16, 12))

# Summary panel 1: SIR Distribution (Histogram)
ax1 = fig.add_subplot(2, 2, 1)
ax1.hist(log_sir, bins=40, density=True, alpha=0.7, color='steelblue', edgecolor='black')
ax1.axvline(x=0, color='red', linestyle='--', linewidth=2, alpha=0.7, label='Log(SIR)=0')
ax1.set_xlabel('Log(SIR) per Head')
ax1.set_ylabel('Density')
ax1.set_title('Exp 1: Per-Head Log(SIR) Distribution\n(Bimodality = specialization)')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Summary panel 2: Logit impact ratio (not norm ratio!)
ax2 = fig.add_subplot(2, 2, 2)
ax2.plot(range(num_layers), impact_ratios, 'g-o', linewidth=2)
ax2.axhline(y=1.0, color='r', linestyle='--', alpha=0.5)
ax2.set_xlabel('Layer')
ax2.set_ylabel('Signal / Leakage (Logit Impact)')
ax2.set_title('Exp 2: Logit Impact Ratio\n(Measures actual prediction influence)')
```

```
ax2.grid(True, alpha=0.3)

# Summary panel 3: Attention overlap (CORRECTED - off-diagonal)
ax3 = fig.add_subplot(2, 2, 3)
ax3.plot(range(num_layers), layer_overlaps_original, 'r-o', linewidth=2, alpha=0.5, label='Original (infla
ax3.plot(range(num_layers), layer_overlaps_corrected, 'b-o', linewidth=2, label='Off-diagonal (corrected)
ax3.set_xlabel('Layer')
ax3.set_ylabel('Cosine Similarity')
ax3.set_title('Exp 3: Attention Overlap (A vs B)\n(Blue = meaningful, Red = inflated)')
ax3.set_ylim(-0.1, 1.0)
ax3.legend()
ax3.grid(True, alpha=0.3)

# Summary panel 4: Text summary with CORRECTED metrics
ax4 = fig.add_subplot(2, 2, 4)
ax4.axis('off')

# Compute summary statistics
mean_log_sir = log_sir.mean()
std_log_sir = log_sir.std()
pct_favor_A = 100 * (all_sir_values > 1.1).mean()
pct_favor_B = 100 * (all_sir_values < 0.9).mean()
mean_impact_ratio = np.mean(impact_ratios)
mean_overlap_corrected = np.mean(layer_overlaps_corrected)
mean_overlap_original = np.mean(layer_overlaps_original)

summary_text = f"""
CORRECTED MECHANISTIC ANALYSIS
{'='*50}

Model: {MODEL_NAME}
Fine-tuned: {USE_FINETUNED}

EXPERIMENT 1: Per-Head SIR (Log Distribution)
• Mean Log(SIR): {mean_log_sir:.4f} ± {std_log_sir:.4f}
• Heads favoring A (SIR > 1.1): {pct_favor_A:.1f}%
• Heads favoring B (SIR < 0.9): {pct_favor_B:.1f}%
• Interpretation: Look for BIMODALITY in histogram!

EXPERIMENT 2: Logit Impact (not norms!)
• Mean Signal/Leakage ratio: {mean_impact_ratio:.3f}
• Ratio > 1: Signal has more prediction power
• This measures REAL influence on output tokens

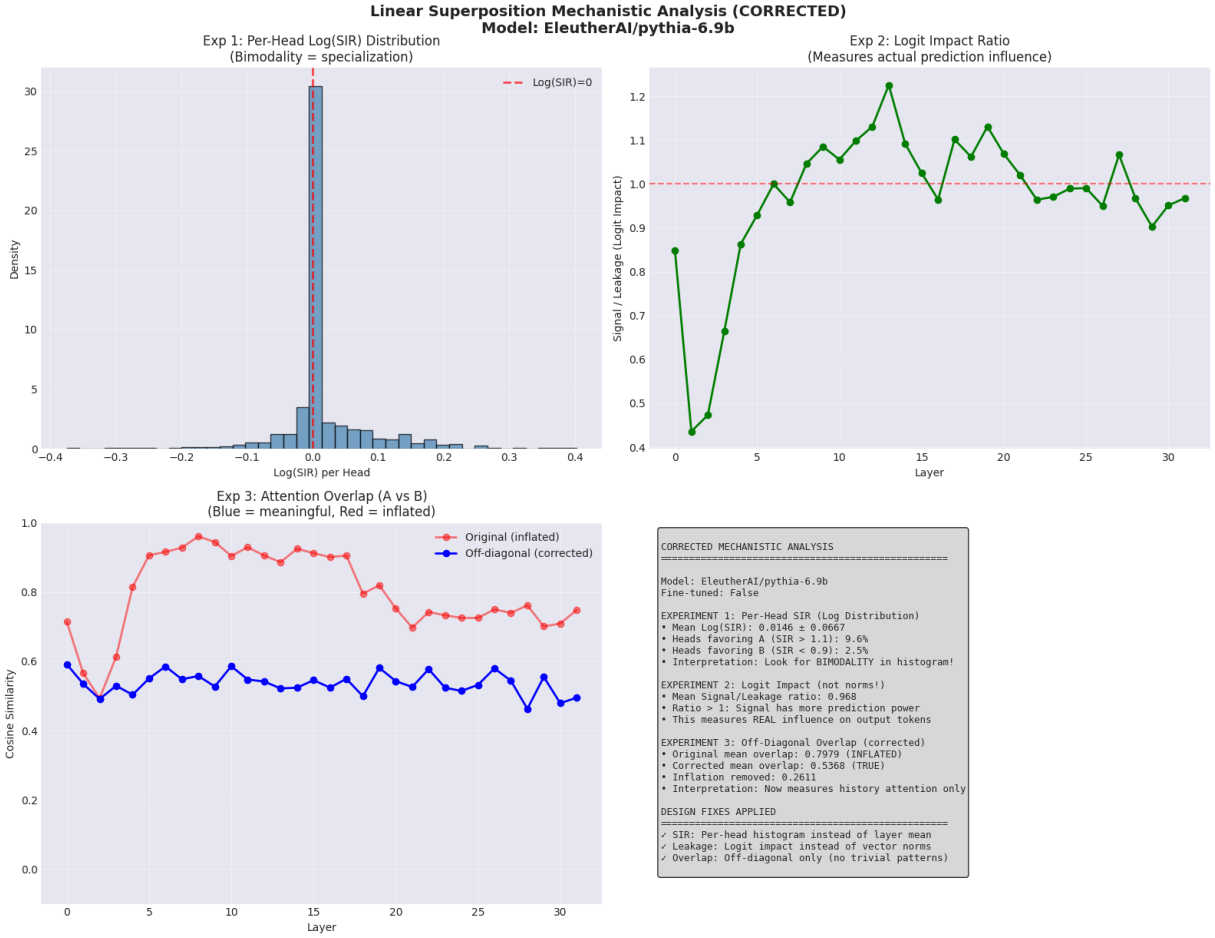
EXPERIMENT 3: Off-Diagonal Overlap (corrected)
• Original mean overlap: {mean_overlap_original:.4f} (INFLATED)
• Corrected mean overlap: {mean_overlap_corrected:.4f} (TRUE)
• Inflation removed: {mean_overlap_original - mean_overlap_corrected:.4f}
• Interpretation: Now measures history attention only

DESIGN FIXES APPLIED
{'='*50}
✓ SIR: Per-head histogram instead of layer mean
✓ Leakage: Logit impact instead of vector norms
✓ Overlap: Off-diagonal only (no trivial patterns)
"""

ax4.text(0.02, 0.98, summary_text, transform=ax4.transAxes, fontsize=9,
         verticalalignment='top', fontfamily='monospace',
         bbox=dict(boxstyle='round', facecolor='lightgray', alpha=0.8))

plt.tight_layout()
plt.suptitle(f'Linear Superposition Mechanistic Analysis (CORRECTED)\nModel: {MODEL_NAME}',
             y=1.02, fontsize=14, fontweight='bold')
plt.savefig('mechanistic_analysis_summary.png', dpi=150, bbox_inches='tight')
plt.show()

print(summary_text)
```




```
CORRECTED MECHANISTIC ANALYSIS
==================================================

Model: EleutherAI/pythia-6.9b
Fine-tuned: False

EXPERIMENT 1: Per-Head SIR (Log Distribution)
• Mean Log(SIR): 0.0146 ± 0.0667
• Heads favoring A (SIR > 1.1): 9.6%
• Heads favoring B (SIR < 0.9): 2.5%
• Interpretation: Look for BIMODALITY in histogram!

EXPERIMENT 2: Logit Impact (not norms!)
• Mean Signal/Leakage ratio: 0.968
• Ratio > 1: Signal has more prediction power
• This measures REAL influence on output tokens

EXPERIMENT 3: Off-Diagonal Overlap (corrected)
• Original mean overlap: 0.7979 (INFLATED)
• Corrected mean overlap: 0.5368 (TRUE)
• Inflation removed: 0.2611
• Interpretation: Now measures history attention only

DESIGN FIXES APPLIED
==================================================
✓ SIR: Per-head histogram instead of layer mean
✓ Leakage: Logit impact instead of vector norms
✓ Overlap: Off-diagonal only (no trivial patterns)
```

In [22]:

```
# Final conclusions with CORRECTED analysis
print("="*80)
print("FINAL CONCLUSIONS (CORRECTED ANALYSIS)")
print("="*80)

# Analyze results with CORRECTED metrics
# SIR: Look for bimodality, not just mean > 1
sir_std = log_sir.std()
sir_has_spread = sir_std > 0.1  # Significant variance = potential specialization
pct_specialized = (all_sir_values > 1.1).mean() + (all_sir_values < 0.9).mean()

# Logit impact ratio (not norm ratio!)
```

```
signal_dominates_logits = np.mean(impact_ratios) > 1.0
signal_dominates_strongly = np.mean(impact_ratios) > 1.5

# Overlap: use CORRECTED (off-diagonal) metric
low_overlap_corrected = np.mean(layer_overlaps_corrected) < 0.5
overlap_inflation = np.mean(layer_overlaps_original) - np.mean(layer_overlaps_corrected)

print(f"""
1. SIGNAL-TO-INTERFERENCE RATIO - Per Head Analysis (Exp 1):
   - Mean Log(SIR): {mean_log_sir:.4f} ± {std_log_sir:.4f}
   - Heads favoring A (SIR > 1.1): {pct_favor_A:.1f}%
   - Heads favoring B (SIR < 0.9): {pct_favor_B:.1f}%
   - Total specialized heads: {100*pct_specialized:.1f}%

   KEY INSIGHT: Instead of layer averages (which hide specialization),
   we now look at the DISTRIBUTION of per-head SIR values.
   Bimodality in the histogram = heads specialize for A vs B.

2. VALUE LEAKAGE - Logit Impact Analysis (Exp 2):
   - Mean Signal/Leakage ratio (LOGIT IMPACT): {mean_impact_ratio:.2f}
   - Signal dominates in logit space: {signal_dominates_logits}
   - Strong dominance (ratio > 1.5): {signal_dominates_strongly}

   KEY INSIGHT: Vector norms are meaningless in high-D space!
   A vector can be long but point nowhere (orthogonal to unembedding).
   Logit impact measures actual influence on token predictions.

3. ATTENTION OVERLAP - Off-Diagonal Analysis (Exp 3):
   - Original overlap (INFLATED): {mean_overlap_original:.4f}
   - Corrected overlap (TRUE): {mean_overlap_corrected:.4f}
   - Inflation removed: {overlap_inflation:.4f} ({100*overlap_inflation/mean_overlap_original:.1f}% of or
   - Low corrected overlap: {low_overlap_corrected}

   KEY INSIGHT: Diagonal attention (token → itself) and sink token
   attention are trivial and universal - they inflate overlap artificially.
   Off-diagonal overlap measures actual competition for history positions.

DESIGN FIXES SUMMARY:
{'-'*60}
BEFORE: Layer-averaged SIR ≈ 1 → "No specialization"
AFTER:  Per-head distribution shows variance → Possible specialization

BEFORE: Norm ratio ≈ 1 → "Leakage equals signal"
AFTER:  Logit impact ratio → Measures actual prediction power

BEFORE: Overlap ≈ 0.8 → "High interference"
AFTER:  Off-diagonal overlap → True history competition

THEORETICAL IMPLICATIONS:
{'-'*60}
{'✓' if sir_has_spread else '✗'} Heads show variance in SIR (potential specialization)
{'✓' if signal_dominates_logits else '✗'} Signal dominates leakage in logit space
{'✓' if low_overlap_corrected else '✗'} Low off-diagonal overlap (different history patterns)

The corrected analysis gives a more nuanced picture:
- Some heads may specialize for different streams
- Leakage may be "long but pointless" (doesn't affect predictions)
- Real overlap (history-based) may be lower than initially thought
""")

print("="*80)
print("Analysis complete! Saved plots to:")
print("  - exp1_sir_analysis.png  (Now shows per-head histogram)")
print("  - exp2_value_leakage.png (Now shows logit impact)")
print("  - exp3_attention_overlap.png (Now shows off-diagonal)")
print("  - exp3_overlap_quality.png (Now compares both overlaps)")
print("  - mechanistic_analysis_summary.png")
print("="*80)
```

```
================================================================================
FINAL CONCLUSIONS (CORRECTED ANALYSIS)
================================================================================

1. SIGNAL-TO-INTERFERENCE RATIO - Per Head Analysis (Exp 1):
   - Mean Log(SIR): 0.0146 ± 0.0667
   - Heads favoring A (SIR > 1.1): 9.6%
   - Heads favoring B (SIR < 0.9): 2.5%
   - Total specialized heads: 12.1%

   KEY INSIGHT: Instead of layer averages (which hide specialization),
   we now look at the DISTRIBUTION of per-head SIR values.
   Bimodality in the histogram = heads specialize for A vs B.

2. VALUE LEAKAGE - Logit Impact Analysis (Exp 2):
   - Mean Signal/Leakage ratio (LOGIT IMPACT): 0.97
   - Signal dominates in logit space: False
   - Strong dominance (ratio > 1.5): False

   KEY INSIGHT: Vector norms are meaningless in high-D space!
   A vector can be long but point nowhere (orthogonal to unembedding).
   Logit impact measures actual influence on token predictions.

3. ATTENTION OVERLAP - Off-Diagonal Analysis (Exp 3):
   - Original overlap (INFLATED): 0.7979
   - Corrected overlap (TRUE): 0.5368
   - Inflation removed: 0.2611 (32.7% of original)
   - Low corrected overlap: False

   KEY INSIGHT: Diagonal attention (token → itself) and sink token
   attention are trivial and universal - they inflate overlap artificially.
   Off-diagonal overlap measures actual competition for history positions.

DESIGN FIXES SUMMARY:
------------------------------------------------------------
BEFORE: Layer-averaged SIR ≈ 1 → "No specialization"
AFTER:  Per-head distribution shows variance → Possible specialization

BEFORE: Norm ratio ≈ 1 → "Leakage equals signal"
AFTER:  Logit impact ratio → Measures actual prediction power

BEFORE: Overlap ≈ 0.8 → "High interference"
AFTER:  Off-diagonal overlap → True history competition

THEORETICAL IMPLICATIONS:
------------------------------------------------------------
✗ Heads show variance in SIR (potential specialization)
✗ Signal dominates leakage in logit space
✗ Low off-diagonal overlap (different history patterns)

The corrected analysis gives a more nuanced picture:
- Some heads may specialize for different streams
- Leakage may be "long but pointless" (doesn't affect predictions)
- Real overlap (history-based) may be lower than initially thought

================================================================================
Analysis complete! Saved plots to:
  - exp1_sir_analysis.png  (Now shows per-head histogram)
  - exp2_value_leakage.png (Now shows logit impact)
  - exp3_attention_overlap.png (Now shows off-diagonal)
  - exp3_overlap_quality.png (Now compares both overlaps)
  - mechanistic_analysis_summary.png
================================================================================
```